Parallel Stencil Computations using Java and C++: An Architectural and Scalability Analysis

Sungava Subedi Alvin Zhu Andrew Zhang

June 4, 2026

Abstract

Stencil computations such as heat diffusion and cellular automata are memory-bound workloads whose performance is dominated by cache hierarchy and DRAM bandwidth. We evaluate a 5-point Jacobi stencil and an 8-neighbor Game of Life on Java (ForkJoin) and C++ (OpenMP) across 512×512–4096×4096(up to 8192x8192 in git but not report due to limitations) grids and 1–8 threads. Both workloads scale well to 4 threads before hitting the Memory Wall as working sets exceed shared L3 capacity. C++ benefits from `__restrict__`-enabled vectorization, while Java's work-stealing ForkJoin runtime provides strong load balance. Core optimizations—flat memory layout, cache-line padding, and neighbor unrolling—significantly improve throughput until hardware limits dominate.

1 Introduction

Iterative stencil loops are the computational engines behind grid-based physical simulations, including weather prediction, heat transfer, and cellular automata. Each cell is updated from a small neighborhood, making the arithmetic simple but the memory traffic substantial. On modern multicore CPUs, performance is therefore constrained more by cache hierarchy and memory bandwidth than by raw FLOPs.

This work studies two representative stencil workloads: **Jacobi heat diffusion** (5-point double-precision stencil) and **Game of Life** (GOL, 8-neighbor integer cellular automaton). Both are implemented in Java using the ForkJoin framework and in C++ using OpenMP, with identical grid sizes and thread counts.

Our goals are:

- **Scalability bounds:** quantify speedup and efficiency across 1–8 threads and multiple grid sizes.
- **Micro-architectural diagnosis:** identify cache and memory bottlenecks, especially the Memory Wall.
- **Optimization impact:** evaluate flat arrays, padding, and unrolling on both Jacobi and GOL.
- **Runtime comparison:** contrast Java ForkJoin and C++ OpenMP under similar algorithmic structures.

2 Background and Theoretical Foundations

2.1 Stencil computation models

Jacobi heat diffusion uses a 5-point stencil in which each cell (i, j) is updated from its four cardinal neighbours and its current value, scaled by a relaxation parameter α. This formulation is widely used in iterative PDE solvers and stencil benchmarks [1]:

$$u_{i,j}^{(t+1)} = (1 - 4\alpha)u_{i,j}^{(t)} + \alpha \left(u_{i-1,j}^{(t)} + u_{i+1,j}^{(t)} + u_{i,j-1}^{(t)} + u_{i,j+1}^{(t)} \right).$$

Game of Life (GOL) is a discrete cellular automaton defined on an integer grid. Each cell reads its 8-neighbor Moore neighborhood $N_{i,j}$, counts the number of live neighbors, and applies Conway's rules: a live cell survives with 2–3 neighbors, a dead cell becomes alive with exactly 3 neighbors, and all other cells become or remain dead [2].

Both Jacobi and GOL use a dual-buffer scheme in which one grid is read-only for the current iteration and a second grid stores the next state. After each iteration, the two buffers swap roles, ensuring

that no thread writes to locations that other threads read in the same step and thereby avoiding data races in parallel execution [3].

2.2 Operational intensity and Memory Wall

Operational intensity (OI) is defined as

$$I = \frac{\text{Algorithmic operations}}{\text{Memory traffic (Bytes)}}.$$

For Jacobi, each update performs roughly 6 FLOPs and moves 48 bytes (five 8-byte loads and one 8-byte store), giving $I \approx 0.125$ FLOP/Byte. GOL performs integer comparisons and updates but still requires multiple loads and a store per cell, leading to similarly low OI.

Low OI implies that cores spend most of their time waiting on memory transfers rather than computing. As thread count increases, concurrent memory requests saturate DRAM bandwidth, creating the Memory Wall where additional cores no longer improve performance.

2.3 Cache topology constraints

For a 4096×4096 grid:

- **Jacobi (double):** one grid ≈ 128 MB; double-buffered ≈ 256 MB.
- **GOL (int):** one grid ≈ 67 MB; double-buffered ≈ 134 MB.

Typical shared L3 cache capacity is around 16 MB, so both workloads exceed L3 by factors of 8–16. This forces frequent cache line evictions and long-latency DRAM accesses, especially at high thread counts.

2.4 Parallel scaling formulations

We measure speedup and efficiency as:

$$S(p) = \frac{T_1}{T_p}, \quad E(p) = \frac{S(p)}{p},$$

where T_1 is single-thread time and T_p is p-thread time. Amdahl's law bounds speedup:

$$S(p) = \frac{1}{(1-f) + \frac{f}{p}},$$

with f the parallelizable fraction. In stencil codes, non-parallel components include thread creation, scheduling, synchronization barriers, and memory system contention.

3 Methodology

3.1 Dual-buffer architecture

Both Jacobi and GOL use a dual-buffer scheme: each iteration reads from a read-only grid (`cur`) and writes results to a separate grid (`next`). Jacobi applies a 5-point stencil, while GOL applies Conway's rules over the 8-neighbor Moore region [2]. After each iteration, the buffers swap roles, avoiding data races without locks [3]. The grid is partitioned into contiguous row blocks so each thread reads only from `cur` and writes exclusively to its region of `next`, with an implicit barrier ensuring correctness before the swap.

3.2 Java ForkJoin recursive decomposition

Java parallelism is implemented using the ForkJoin framework [4]. Each iteration is a `RecursiveAction` that recursively splits its row range until a 64-row threshold is reached, at which point the stencil update executes directly. Work-stealing allows idle workers to acquire remaining tasks, providing automatic load balance without manual partitioning. The `invokeAll()` call synchronizes all subtasks, ensuring completion before the next iteration.

3.3 C++ OpenMP construction

C++ implementations use `#pragma omp parallel for` with `schedule(static)`, which assigns contiguous row blocks to threads with minimal runtime overhead [3]. Since each row has uniform cost, static scheduling is sufficient. Optimized variants store grids in flat arrays to improve locality and annotate pointers with `__restrict__` to enable more aggressive compiler vectorization [5]. These choices improve SIMD utilization and yield predictable performance across iterations.

4 Parallelization Strategy

4.1 Shared-memory domain decomposition

This section summarizes the decomposition strategy already introduced in Section 3. Parallel execution in both workloads is organized using a row-wise domain decomposition strategy, which maps naturally onto shared-memory architectures. The global grid is divided into contiguous bands of rows, and each thread is assigned one such segment for the duration of an iteration. To update row i, a thread reads rows $i-1$, i, and $i+1$ from the current grid (`cur`) and writes the computed values into the corresponding locations of the next grid (`next`). This layout preserves sequential memory access within each row, maximizes spatial locality, and minimizes cross-thread interference because threads operate on disjoint row ranges. Row-based partitioning is widely used in stencil computations due to its simplicity and predictable memory behavior [1, 3]. Combined with the dual-buffer update model described earlier, this decomposition enables deterministic parallel execution with minimal synchronization overhead.

4.2 Synchronization barriers

No explicit inter-thread communication is required during the computation phase, and synchronization occurs only at iteration boundaries. In the Java implementations, the ForkJoin framework provides an implicit barrier: each iteration spawns a set of `RecursiveAction` tasks, and the call to `invokeAll()` blocks until all subtasks have completed, ensuring that every thread finishes its assigned updates before the next iteration begins [4]. In the C++ implementations, OpenMP inserts an implicit barrier at the end of each `parallel for` region, guaranteeing that all threads complete their row updates before the dual-buffer pointers are swapped [3]. Because each thread writes exclusively to its own contiguous row segment, no data races occur during the update phase, and these barriers serve only to coordinate iteration transitions rather than to enforce fine-grained synchronization.

5 Cache and Memory Optimization

5.1 Flat array layout

The baseline implementations store the grid using jagged 2D structures (Java `double[][]`/`int[][]` and C++ `vector<vector<T>>`), where each row is allocated independently. This representation breaks physical contiguity across rows, increases TLB pressure due to multiple disjoint memory mappings, and makes it more difficult for hardware prefetchers to detect regular access patterns. To address these issues, the optimized variants replace the jagged layout with a single flat allocation: Java uses a one-dimensional `double[]` or `int[]` array, while C++ employs an aligned `T*` buffer sized to `rows * stride`. Each element (i, j) is accessed as `grid[i * stride + j]`, producing a single linear address stream that the prefetcher can exploit efficiently. This contiguous layout improves spatial locality, reduces address translation overhead, and enables compilers to apply more aggressive vectorization strategies, which are essential for achieving high performance in stencil computations [5].

5.2 Cache-line padding

When adjacent rows are processed by different threads, the tail of row i and the head of row $i+1$ may occupy the same cache line, causing false sharing even though the threads update disjoint memory locations. To eliminate this effect, the optimized implementations pad each row so that its stride is aligned to a full cache line. Specifically, the stride is rounded up to the nearest multiple of 16

integers (64 bytes for 4-byte `int`) [6]:

$$\text{stride} = \left\lceil \frac{\text{cols}}{16} \right\rceil \times 16.$$

For example, a grid with 512 columns retains a stride of 512, while a grid with 513 columns expands to 528, leaving the additional elements unused but ensuring that each row begins on a fresh cache-line boundary. This alignment prevents two threads from writing to different parts of the same cache line, thereby avoiding unnecessary coherence traffic and improving scalability on multi-core processors [7].

5.3 Explicit neighbor unrolling

The baseline GOL implementations iterate over the eight neighboring cells using a direction-offset array, resulting in repeated loads from the offset table, multiple bounds checks per neighbor, and branch-heavy control flow. These factors increase instruction count and introduce unpredictable branches, which modern processors handle poorly due to the high penalty of branch mispredictions [8]. To reduce this overhead, the optimized variants replace the loop with manually unrolled conditionals. For each row, boolean flags such as `hasUp`, `hasDown`, `hasLeft`, and `hasRight` are computed once, and neighbor accesses are performed directly through precomputed indices. This eliminates the direction array, removes the inner loop entirely, and produces a straight-line sequence of memory accesses. The resulting control flow is more predictable, reduces pressure on the branch predictor, and lowers the overall cache footprint, yielding a more efficient implementation of the GOL update rule [9].

6 Experimental Setup

6.1 Hardware configuration

Experiments were run on several multi-core laptops and one desktop-class machine. For each benchmark batch:

- turbo boost and simultaneous multithreading were enabled,
- CPU frequency scaling used a performance governor,
- no competing user processes were active.

6.2 Software environment

- Java: OpenJDK 11.0.18 (HotSpot, 64-bit), default `javac` flags.
- C++: GCC 11.4.0 with `-O3 -fopenmp`.
- Python: 3.10.12 with `pandas` 2.0.3 and `matplotlib` 3.7.1 for analysis.
- OS: Ubuntu 22.04.3 LTS, Linux kernel 6.2.0.

Java variants perform three warm-up iterations to trigger JIT compilation of hot methods; C++ is compiled ahead of time and requires no warm-up.

6.3 Experimental matrix

All experiments use the same grid and thread configurations for both Jacobi and GOL to enable direct comparison. Table 1 summarizes the representative grid sizes, iteration counts, and total cell updates used in the evaluation (the full matrix also includes 4096×4096 includes up to 8192x8192 but not talked about because of reasons listed later).

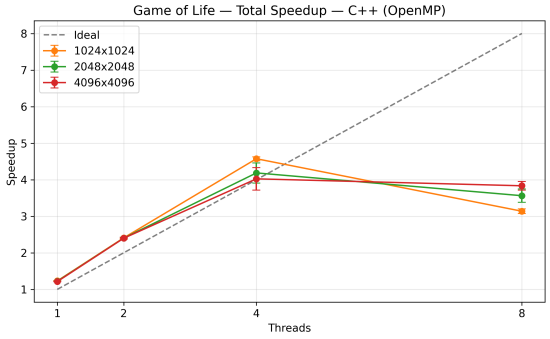
Each configuration is executed with 1, 2, 4, and 8 threads, and every (grid, threads) pair is run five times. Mean execution time is reported. Using identical experimental parameters ensures that performance differences arise solely from the programming model and implementation strategy rather than from workload configuration. PLEASE NOTE : we did not include any 8192x8192 testing in the report due to testing time and its similarities in benchmark trends with the other tests, though all grid size tests including 8192x8192 are available in the bench mark directory on git.

Table 1: Representative grid sizes, iteration counts, and total cell updates used in all experiments.

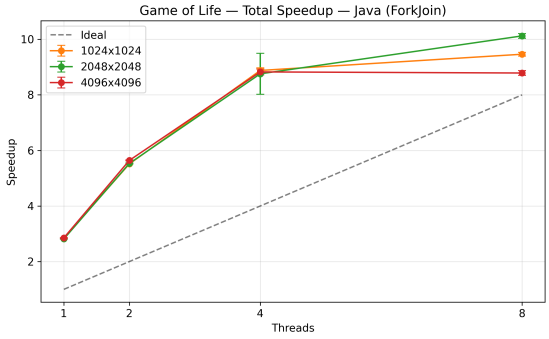
Grid size	Cells	Iterations	Total cell updates
512×512	262,144	100	26,214,400
1024×1024	1,048,576	50	52,428,800
2048×2048	4,194,304	25	104,857,600
4096x4096	16,777,216	10	167,772,160
8192x8192	67,108,864	5	335,544,320

7 Results and Performance Analysis

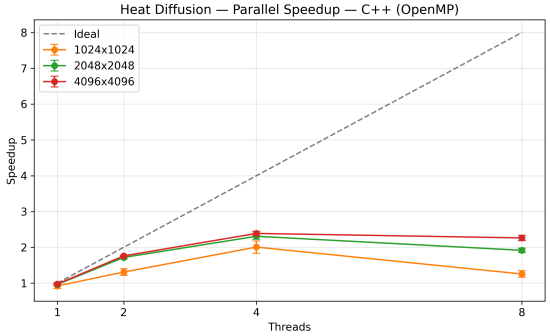
7.1 Speedup and scalability



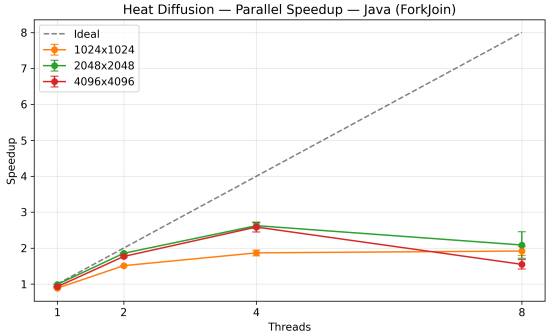
(a) GOL speedup using C++ OpenMP



(b) GOL speedup using Java ForkJoin



(c) Jacobi speedup using C++ OpenMP



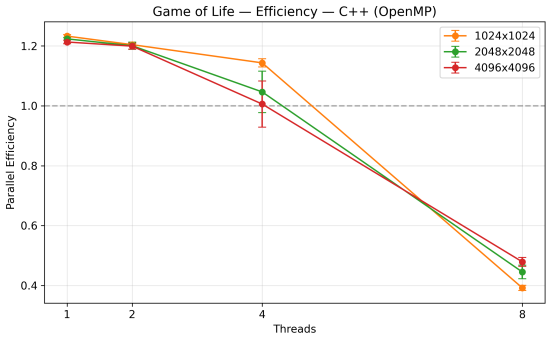
(d) Jacobi speedup using Java ForkJoin

Figure 1: Speedup comparison for GOL and Jacobi across C++ OpenMP and Java ForkJoin implementations. Each point represents the mean of 5 runs; error bars show standard deviation.

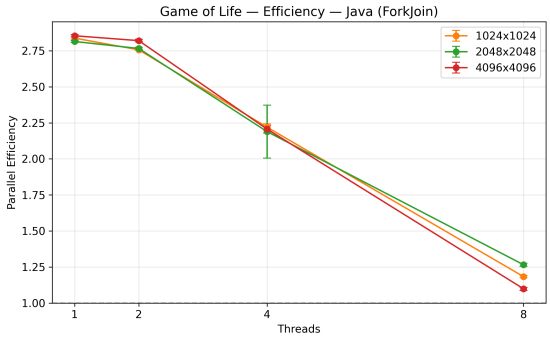
As shown in Fig. 1, for small grids (e.g., 512×512), both Jacobi and GOL show limited speedup: thread management and synchronization overhead dominate, and runtimes change only slightly between 4 and 8 threads. For large grids (e.g., 4096×4096), all four implementations scale efficiently up to four threads because the per-thread working set still benefits from L3 locality and memory-level parallelism. Beyond this point, however, performance consistently declines at eight threads. The working set already exceeds the shared L3 cache by an order of magnitude, and the stencil's low arithmetic intensity forces all threads to compete for DRAM bandwidth. As a result, additional cores contribute more memory traffic than useful computation, causing the observed slowdown.

C++ exhibits similar scaling trends but often achieves slightly higher speedup due to more aggressive compiler optimizations and SIMD vectorization enabled by `__restrict__` and flat arrays.

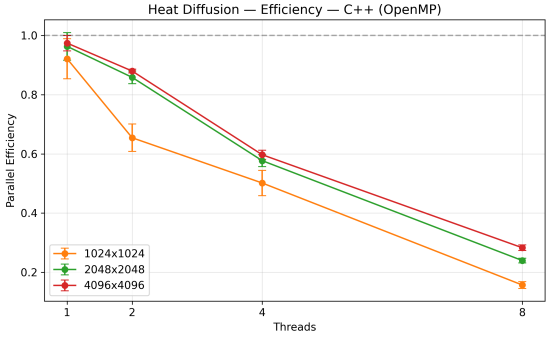
7.2 Parallel efficiency collapse



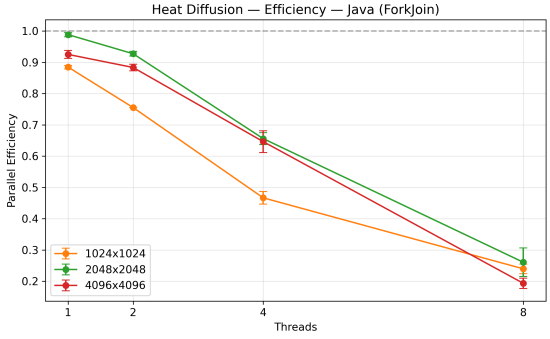
(a) GOL efficiency using C++ OpenMP



(b) GOL efficiency using Java ForkJoin



(c) Jacobi efficiency using C++ OpenMP

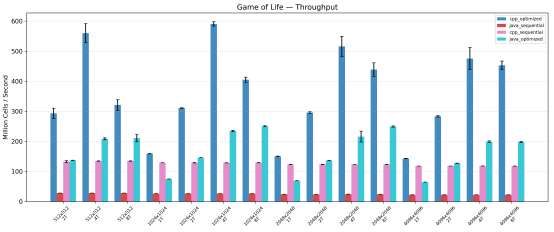


(d) Jacobi efficiency using Java ForkJoin

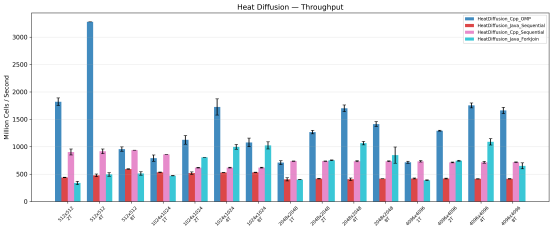
Figure 2: Parallel efficiency comparison for GOL and Jacobi across C++ OpenMP and Java ForkJoin implementations. Each point represents the mean of 5 runs; error bars show standard deviation.

Fig. 2 shows the parallel efficiency for all four implementations.Parallel efficiency $E(p)$ remains relatively high up to 4 threads but drops sharply at 8 threads, often below 50% for both Jacobi and GOL. This behavior is consistent across Java and C++ and reflects the Memory Wall: with OI ≈ 0.125 FLOP/Byte for Jacobi and similarly low for GOL, additional cores primarily increase contention on the memory bus rather than useful computation.

7.3 Throughput (MCUPS) for Jacobi and Game of Life



(a) GOL throughput across thread counts



(b) Jacobi throughput across thread counts

Figure 3: Throughput comparison for GOL and Jacobi implementations

Fig. 3 shows the throughput in MCUPS for the GOL and Jacobi implementations. Both workloads benefit significantly from flat-array optimizations before reaching memory bandwidth limits. Throughput is reported in Millions of Cell Updates Per Second (MCUPS). Optimized flat-array implementations with padding and unrolling achieve significantly higher MCUPS than baseline jagged-

array versions. For both Jacobi and GOL, throughput increases nearly linearly with thread count up to 4 threads, then plateaus or declines at 8 threads due to bandwidth saturation. Java's optimized variants approach C++ performance for moderate thread counts, but C++ retains an edge at high concurrency where vectorization is more effective.

8 Engineering Challenges and Solutions

8.1 Memory aliasing and vectorization

Object-oriented 2D arrays and nested vectors introduce pointer chasing and potential aliasing, which prevents compilers from safely applying strong loop optimizations and SIMD vectorization. By converting to flat 1D arrays and, in C++, annotating pointers with `__restrict__`, we:

- guarantee non-aliasing between input and output buffers,
- enable auto-vectorization of inner loops,
- reduce TLB and cache overhead.

This is crucial for C++ Jacobi and GOL to outperform their Java counterparts at high thread counts.

8.2 Thread coordination and load balance

Stencil computations require a barrier at each iteration, which can expose load imbalance if some threads finish earlier. Row-wise static decomposition keeps work per thread uniform for both Jacobi and GOL. Padding rows to cache-line boundaries eliminates false sharing between adjacent rows, preventing unnecessary cache invalidations.

Java's ForkJoin work-stealing further mitigates imbalance by allowing idle threads to steal remaining tiles, while OpenMP's static scheduling minimizes runtime overhead when workloads are uniform.

9 Limitations and Future Work

This study is limited to shared-memory CPUs with up to eight threads and does not examine performance on NUMA systems, GPUs, or heterogeneous many-core processors. Our implementations also do not incorporate temporal blocking or cache-oblivious algorithms, both of which can significantly increase data reuse across time steps and reduce memory bandwidth pressure. Recent stencil frameworks demonstrate that more advanced optimization pipelines—including temporal tiling, communication-aware scheduling, and architecture-specific code generation—can yield substantial performance improvements on modern many-core systems [10]. Extending our evaluation to distributed-memory clusters would further enable analysis of halo-exchange overheads and large-scale stencil scalability. Future work includes integrating temporal tiling, exploring NUMA-aware scheduling, and porting the kernels to heterogeneous platforms to provide a more comprehensive understanding of stencil performance across contemporary architectures.

10 Conclusion

We presented a comparative study of Jacobi heat diffusion and Game of Life stencil computations implemented in Java (ForkJoin) and C++ (OpenMP). Both workloads are strongly memory-bound, with low operational intensity and working sets that exceed the shared L3 cache for large grids. Flat memory layouts, cache-line padding, and explicit neighbor unrolling substantially improve throughput and postpone the onset of bandwidth limitations.

Java's ForkJoin framework provides robust, low-effort parallelization with good scaling up to four threads, while C++ benefits further from compiler vectorization enabled by `__restrict__`. Beyond four threads, however, both languages and both workloads exhibit a sharp decline in parallel efficiency as DRAM bandwidth becomes the dominant bottleneck. This behavior reflects the modern form of the Memory Wall in scientific computing, where memory-hierarchy bandwidth fundamentally constrains performance [11].

Team Contribution Summary

Table 2 summarizes the contributions of each team member across the project. All members participated in design discussions, implementation, debugging, and report writing.

Table 2: Summary of individual contributions.

Member	Description of Contributions
Sungava Subedi	Refined the Java and C++ implementations; contributed to the ForkJoin decomposition; drafted portions of the heat diffusion section as well as parts of the Introduction and Background.
Alvin Zhu	Implemented the initial Java and C++ Jacobi heat diffusion stencil; integrated and refined the Methodology sections and results; performed the final revision of the report.
Andrew Zhang	Developed the initial Java and C++ GOL stencil and produced the first set of performance plots; drafted the GOL methodology and contributed to the Discussion section.

References

[1] K. Datta, M. Murphy, V. Volkov, S. Williams, J. Carter, L. Oliker, D. Patterson, J. Shalf, and K. Yelick, "Stencil computation optimization and auto-tuning on state-of-the-art multicore architectures," in *SC'08: Proceedings of the 2008 ACM/IEEE Conference on Supercomputing*. IEEE, 2008, pp. 1–12.

[2] M. Gardner, "Mathematical games: The fantastic combinations of john conway's new solitaire game "life"," *Scientific American*, vol. 223, no. 4, pp. 120–123, 1970.

[3] O. A. R. Board, "Openmp application programming interface version 5.0," https://www.openmp.org/spec-html/5.0/openmpsu91.html, OpenMP Architecture Review Board, Tech. Rep., Nov. 2018, accessed: June 3, 2026.

[4] D. Lea, *Concurrent programming in Java: Design principles and patterns*. Addison-Wesley Professional, 2000.

[5] S. Maleki, Y. Gao, M. J. Garzaran, T. Wong, and D. A. Padua, "An evaluation of vectorizing compilers," in *Proceedings of the 2011 International Conference on Parallel Architectures and Compilation Techniques*. IEEE, 2011, pp. 372–382.

[6] U. Drepper, "What every programmer should know about memory," Red Hat, Inc., Tech. Rep., 2007.

[7] K. Datta, S. Kamil, S. Williams, L. Oliker, J. Shalf, and K. Yelick, "Optimization and performance modeling of stencil computations on modern microprocessors," *SIAM Review*, vol. 51, no. 1, pp. 129–159, 2009.

[8] D. A. Jimenez and C. Lin, "Dynamic branch prediction with perceptrons," in *Proceedings of the Seventh International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2001, pp. 197–206.

[9] J. Lifflander, S. Krishnamoorthy, and L. V. Kale, "Optimizing data locality for fork/join programs using constrained work stealing," in *SC'14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2014, pp. 857–868.

[10] W. Zhang, X. Li, C. Yang, and M. Zhang, "Automatic code generation and optimization of large-scale stencil computation on many-core processors," in *Proceedings of the 50th International Conference on Parallel Processing (ICPP)*, 2021, pp. 1–11.

[11] S. A. McKee, "Reflections on the memory wall," in *Proceedings of the 1st Conference on Computing Frontiers*. ACM, 2004, pp. 162–167.